Skill–Opportunity Gap Analysis: Project Report

Hackathon Submission | October 2026

1. Executive Summary

This project is a decision-support system that matches candidates to opportunities (jobs) and tells them exactly why they qualify, why they do not, and what they need to learn to close the gap. Its defining design choice is a strict **two-stage pipeline**: Stage 1 decides eligibility using deterministic rules only, and Stage 2 uses AI/NLP solely to rank the opportunities that already passed. AI can never make an ineligible opportunity appear eligible.

The system recognizes synonymous skills (for example, "ReactJS" and "React"), explains every ranking and every rejection in plain English, detects missing skills, builds step-by-step learning pathways from a local static course catalogue, and recomputes everything instantly when the candidate edits their profile. It runs fully offline.

2. Problem Statement

Candidates struggle to know which opportunities they can actually apply to, how well they fit, and what to do about gaps. Existing tools tend to blend eligibility and relevance into a single opaque score, which produces two failures: candidates are shown opportunities they are not allowed to apply for, and rejections come with no explanation.

The challenge is to build a system that separates hard eligibility from preference ranking, explains its outputs, and guides candidates toward closing skill gaps, while treating the opportunity's published criteria as the single source of truth.

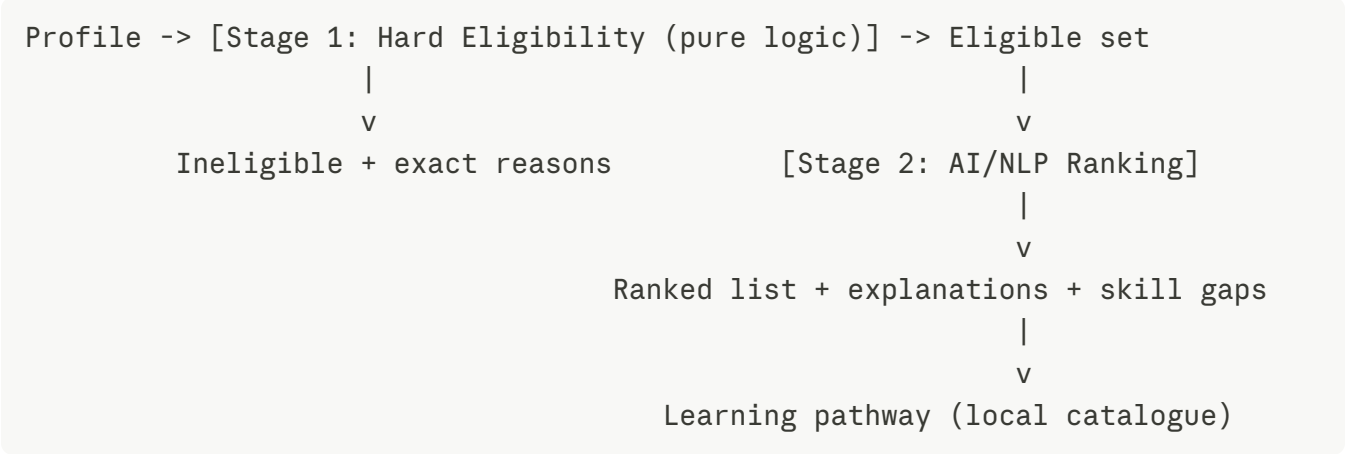
3. Objectives and Mandatory Requirements

#	Requirement	Summary
1	Two-stage filtering	Hard eligibility is separate from preference ranking

#	Requirement	Summary
2	Strict disqualification	Ineligible opportunities never appear as eligible, regardless of skill match
3	Eligible-only ranking	Only opportunities that pass all hard criteria are ranked
4	Semantic skill matching	Synonyms such as "ReactJS" = "React" are recognized using AI/NLP
5	No AI for eligibility	AI scores relevance only; strict logic decides eligibility
6	Explainable matches	Plain-English reason for each opportunity's rank
7	Explainable failures	The exact failed condition is stated (e.g., "Requires 2+ years experience")
8	Skill gap detection	Exact missing skills are identified per opportunity
9	Offline learning pathways	Step-by-step courses built from a local static catalogue only
10	Real-time updates	Profile edits immediately re-run filters and refresh the UI
11	Decision support role	Recommendations guide; opportunity criteria remain the source of truth

4. System Architecture

The pipeline flows in one direction so that no later stage can alter an earlier decision.



Layer	Responsibility	Uses AI?
Data layer	JSON/SQLite store for opportunities, courses, skill taxonomy	No
Eligibility engine	Deterministic rule checks	No
Skill normalizer	Synonym and alias resolution	Yes (offline embeddings)
Ranking engine	Relevance scoring of eligible items only	Yes
Gap analyzer	Required skills minus owned skills	Logic (set operations)
Pathway builder	Course sequencing from local catalogue	No (graph logic)
UI and state	Reactive profile editing and instant recompute	No

5. Data Models

Candidate profile holds skills, experience in years, location, education level, certifications, and availability.

Opportunity is the single source of truth. It separates `hard_criteria` (minimum experience, allowed locations, minimum education, mandatory certifications, deadline) from `skills_required` and `skills_preferred`. Skills are never treated as hard criteria unless explicitly modeled as a mandatory certification.

Course catalogue entries contain an id, title, the skills the course teaches, prerequisites, duration in hours, and level.

Skill taxonomy lists canonical skills with their aliases and optional related-skill links, for example `react` with aliases `ReactJS`, `React.js`.

Keeping hard criteria and skills in separate fields is what makes strict disqualification enforceable by construction.

6. Stage 1: Hard Eligibility (No AI)

Each criterion is a small pure function returning a pass/fail result and a human-readable reason. The engine checks:

- Minimum years of experience
- Allowed locations
- Minimum education level (compared using an ordered rank table)
- Mandatory certifications
- Application deadline

Design rules

- All checks run without short-circuiting, so the candidate sees every failed condition at once.
- The output is binary: eligible, or ineligible with a list of reasons.
- Ineligible opportunities are removed from the ranking pipeline entirely, so they cannot re-enter through a high skill score.
- Missing profile data is treated as "unknown" and the user is prompted to supply it, rather than silently passing or failing.

Because this stage contains no machine learning, its behavior is fully predictable, testable, and auditable.

7. Stage 2: Semantic Skill Matching and Ranking

7.1 Skill normalization

Skills are resolved to canonical forms in two steps:

1. **Alias dictionary (exact):** "ReactJS", "React.js", and "React" all map to `react`. This also guards against dangerous conflations such as "Java" versus "JavaScript".
2. **Embedding similarity (fallback):** For unknown terms, a small local sentence-embedding model (such as all-MiniLM-L6-v2) finds the nearest canonical skill. A match is accepted only if cosine similarity meets a conservative threshold (about 0.80); otherwise the term is kept as-is.

Canonical embeddings are pre-computed at startup, so the system works offline and responds quickly.

7.2 Relevance scoring (eligible opportunities only)

```
score = 0.60 * required_skill_coverage
      + 0.20 * preferred_skill_coverage
      + 0.10 * semantic_similarity(profile_summary, opportunity_description)
      + 0.10 * related_skill_bonus
```

- Required coverage is matched required skills divided by total required skills.
- The related-skill bonus gives partial credit for adjacent skills from the taxonomy.
- Weights are stored in a configuration file so they can be justified and tuned.
- Results are sorted in descending order, with ties broken by deadline or experience fit.

The score influences order only. It has no power over eligibility.

8. Explainability

All explanations are generated from structured results using string templates, not free-form language generation. This guarantees that the text always reflects what the logic actually computed.

Why an opportunity ranks where it does:

> "Ranked #2: You match 3 of 4 required skills (React, JavaScript, Git) and 1 preferred skill. Missing: TypeScript. Your location (Pune) and education (B.Tech) meet the requirements."

Why an opportunity is not eligible:

> "Not eligible: Requires 2+ years experience (you have 1.5). Location must be Bangalore or Remote (yours is Pune)."

9. Skill Gap Detection

For each opportunity of interest, the profile's canonical skills are compared with the opportunity's canonical required and preferred skills:

- **Matched:** required skills the candidate already has
- **Missing required:** required skills the candidate lacks
- **Missing preferred:** optional skills that would strengthen the application

- **Close gaps:** missing skills where the candidate holds a related skill (for example, knows JavaScript and needs TypeScript), identified through taxonomy links

10. Offline Learning Pathway Builder

Pathways are built only from the local static catalogue, with no live API calls.

1. Map each missing skill to courses whose `teaches` field contains it.
2. Build a prerequisite directed acyclic graph (DAG) from course prerequisites.
3. Remove prerequisites the candidate already satisfies.
4. Topologically sort the remaining courses into an ordered path.
5. When several courses teach the same skill, prefer the shortest or lowest-level one.
6. Sum durations to give a total time estimate.

Example: Missing TypeScript and Docker produces: (1) TypeScript Basics, 6 hours; (2) Docker Fundamentals, 4 hours; total 10 hours.

If the catalogue has no course for a skill, the system says so explicitly ("No course available in catalogue for X") and never fabricates one.

11. Real-Time Dynamic Updates

The UI is driven by a reactive state store:

```
Profile edit -> state update -> eligibility -> ranking -> gaps -> pathways ->
UI re-render
```

Edits to skills, location, education, experience, or certifications trigger a full pipeline recompute, debounced by roughly 300 ms. Skill embeddings and canonical mappings are cached, so only new inputs reach the model. With a few hundred opportunities, recomputation completes in milliseconds.

12. Decision-Support Behavior

- The interface shows three views: **Eligible (ranked)**, **Not Eligible (with exact reasons)**, and **Gap and Pathway** per opportunity.
- Recommendations are phrased as guidance (for example, "Strong fit: closing one gap improves your match"), never as overrides.

- Opportunity criteria are read-only; the system never edits or relaxes them.
- A **what-if view** shows how profile changes would unlock opportunities, for example: "With 0.5 more years of experience, 2 more roles become eligible."

13. Technology Stack

Component	Choice
Frontend	React with Tailwind and Zustand (or Streamlit for rapid prototyping)
Backend	FastAPI (Python) with a `POST /analyze` endpoint
NLP	sentence-transformers (local model) plus alias dictionary
Data	JSON files or SQLite
Graph logic	networkx for the course prerequisite DAG

14. Requirement Traceability

Requirement	How it is satisfied
Two-stage filtering	Eligibility and ranking are separate modules with a one-way data flow
Strict disqualification	Ineligible items are dropped before ranking begins
Eligible-only ranking	The ranker accepts only the eligible list as input
Semantic skill matching	Alias map plus local embeddings with a conservative threshold
No AI for eligibility	Stage 1 consists of plain conditional logic
Explainable matches	Templates populated from real score components
Explainable failures	One reason string per failed criterion
Skill gap detection	Set difference on canonical skills

Requirement	How it is satisfied
Offline pathways	Local catalogue, prerequisite DAG, topological sort
Real-time updates	Reactive state triggers the full pipeline on every edit
Decision support	Criteria are read-only; output is advisory

15. Testing Strategy

- **Eligibility unit tests:** every criterion tested at its boundary (for example, exactly 2.0 years against a 2-year minimum).
- **Invariant test:** for randomized profiles, assert that no ineligible opportunity ever appears in the ranked list.
- **Synonym tests:** a table of aliases ("ReactJS", "React.js", "react") must resolve identically; "Java" must never match "JavaScript".
- **Determinism test:** the same profile must always yield the same ranking and explanations.
- **Pathway tests:** verify prerequisite ordering, removal of already-owned prerequisites, and the "no course available" message.
- **Latency test:** profile edit to updated UI under 500 ms on the demo dataset.

16. Edge Cases and Limitations

- **Missing profile fields:** shown as "unknown" with a prompt to complete them, never silently treated as a pass or fail.
- **Ambiguous skills:** resolved by the alias dictionary before any embedding lookup.
- **Embedding false positives:** mitigated by a conservative threshold and a manual alias override.
- **Static catalogue:** pathways are limited to courses in the local catalogue; the system reports gaps in coverage rather than guessing.
- **Scoring weights:** the weights are heuristic and configurable; they affect ordering only and do not claim predictive accuracy.

17. Future Work

- Weighted or tiered skill proficiency levels instead of binary have/lack.
- Candidate feedback loop to tune ranking weights.
- Bulk import of opportunity feeds with automated schema validation.
- Multi-language skill aliases.
- Exportable gap-analysis report per candidate.

18. Conclusion

By separating deterministic eligibility from AI-driven ranking, this system gives candidates results they can trust: nothing ineligible is ever shown as a match, every outcome is explained in plain English, and every gap comes with an actionable, offline learning path. The opportunity's own criteria remain the single source of truth, and the AI's role is limited to helping candidates see which of their valid options fit best.